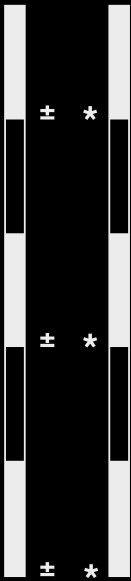


BITTY VAULT: SECURITY REVIEW REPORT



Contents

1	Introduction	3
2	About Verilog Solutions	4
3	Service Scope	5
3.1	Service Stages	5
3.2	Methodology	5
3.3	Audit Scope	5
3.4	Out of Scope	6
4	Project Summary	7
4.1	Roles	7
4.2	Architecture	7
4.3	Security Model (as designed)	7
5	Findings and Improvement Suggestions	8
5.1	Critical	8
5.2	High	9
5.3	Medium	14
5.4	Low	27
5.5	Informational	40
5.6	Positive Security Properties Observed	51
6	Appendix	52
6.1	Appendix I: Severity Categories	52
6.2	Appendix II: Status Categories	52
7	Disclaimer	53

1 | Introduction

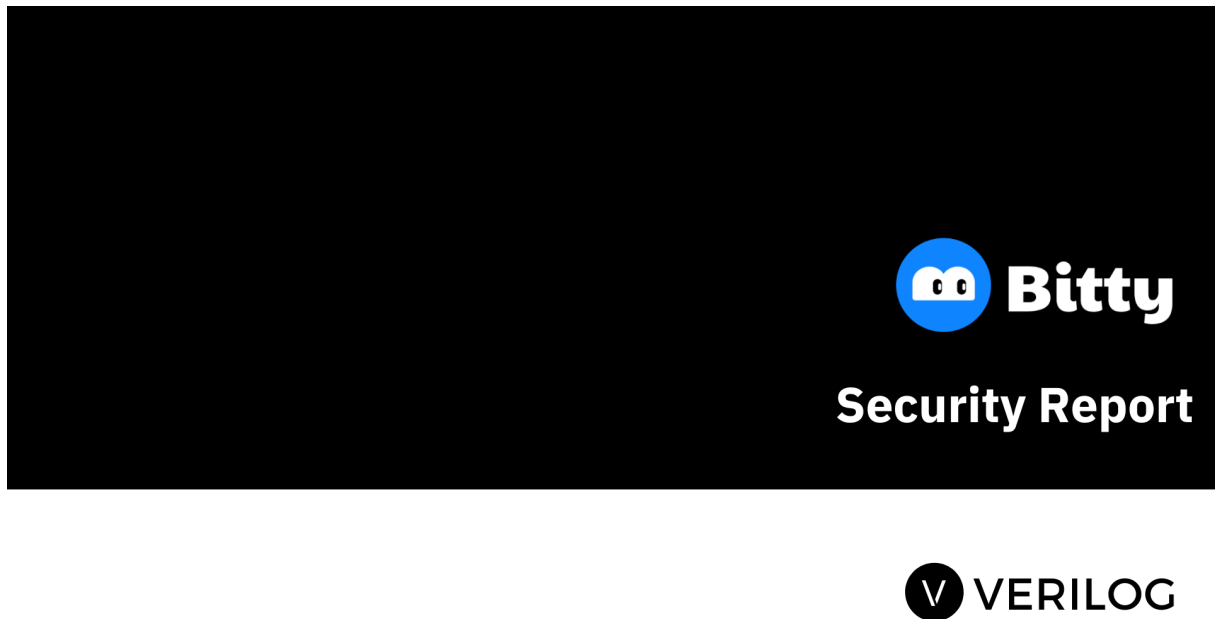


Figure 1.1: BittyIO Vault Report Cover

This report presents our engineering engagement with the BittyIO team on their Ethereum smart contracts security analysis across the **guard**, **vault**, and **protocol-store** repositories.

The review was conducted in multiple iterative rounds beginning on 2026-07-06, with client remediation feedback, follow-up findings (2026-07-13), and a second-pass review (2026-07-21). Final statuses in this report are verified against the latest in-scope commits listed in Section 3.

A substantial portion of the initial findings were addressed not only by targeted patches but by subsequent architecture improvements (role-separated vault facets, **RiskConfig** / risk levels, intent-only trading, content-hash payout approvals, invest-cap and trade-limit accounting, and factory activation redesign). Client confirmation of the final **vault** and **protocol-store** commits is reflected in the scope table.

2 | About Verilog Solutions

Founded by a group of cryptography researchers and smart contract engineers in North America, Verilog Solutions elevates the security standards for Web3 ecosystems by being a full-stack Web3 security firm covering smart contract security, consensus security, and operational security for Web3 projects.

Verilog Solutions team works closely with major ecosystems and Web3 projects and applies a quality above quantity approach with a continuous security model. Verilog Solutions onboards the best and most innovative projects and provides the best-in-class advisory services on security needs, including on-chain and off-chain components.

3 | Service Scope

3.1 | Service Stages

Our auditing service includes the following two stages:

- Pre-Audit Consulting Service
- Smart Contract Auditing Service

3.1.1 | Pre-Audit Consulting Service

As a part of the audit service, the Verilog Solutions team worked closely with the BittyIO development team to discuss potential vulnerabilities and smart contract development best practices in a timely fashion. Findings were exchanged promptly across multiple review rounds; remediations and design responses were reviewed against updated commits.

3.1.2 | Smart Contract Auditing Service

The Verilog Solutions team analyzed the in-scope repositories using a detail-oriented approach to capture the fundamental custody, allowlist, DeFi adapter, and payout logic, and suggested improvements to the existing code. Details can be found under Findings And Improvement Suggestions.

3.2 | Methodology

■ Code Assessment

- We evaluate the overall quality of the code and comments as well as the architecture of the repositories.
- We help the project development team improve the overall quality of the repositories by providing suggestions on refactorization to follow the best practices of Web3 software engineering.

■ Code Logic Analysis

- We dive into the data structures and algorithms in the repositories and provide suggestions to improve correctness, access control, and fund-safety invariants.
- We analyze the hierarchy among modules (factory, vault, guard, protocol adapters) and the relations among source files, including role separation and protocol lifecycle handling.

3.3 | Audit Scope

Our auditing for BittyIO covered production Solidity under `src/` in the following repositories:

- **guard** — <https://github.com/BittyIO/guard>
Global allowlist and deprecation registry for assets and protocols (`BittyV1Guard.sol`).
- **vault** — <https://github.com/BittyIO/vault>
Vault factory, per-owner vault clones, custody/payout logic, and asset-manager DeFi facet/libraries.
- **protocol-store** — <https://github.com/BittyIO/protocol-store>
Protocol adapters for Aave V3, Lido V2, Sky V1, Uniswap V3, and CoW Swap.

3.3.1 | Commits Reviewed

Repo	Branch	Commit
guard (initial)	main	6b83447
vault (initial)	develop	fabf831
protocol-store (initial)	main	bf9840b
guard (follow-up)	main	4a109d3
vault (follow-up)	develop	a63638c
protocol-store (follow-up)	main	c0cbd2b
guard (final)	main	4a109d3d7034f71b6338f8c022dd085e7daa2bea
vault (final)	develop	f40b56b89076c8416c99a783b6a2449e59fea7b7
protocol-store (final)	main	c07dbc8b9ee9b558da630afe094f9722d7737024

3.3.2 | In-Scope Size (final)

Area	~LOC (src/)
vault	~4,100
guard	~550
protocol-store	~2,200
Total	~6,850

Dependencies (OpenZeppelin, Solmate, and similar libraries) and test suites are out of scope.

3.4 | Out of Scope

- Live deployment keys and multisig / operational security configuration
- Frontend and off-chain agent / AI asset-manager security
- CoW API posting and off-chain EIP-1271 integration flows
- Formal verification and invariant fuzzing campaigns
- Economic modeling of LP / yield strategies

4 | Project Summary

Bitty Vault is a role-separated treasury system for Ethereum. Each user activates a deterministic vault clone; Bitty Guard supplies a global allowlist of assets and protocols; Bitty Protocol Store provides adapters for lending, staking, AMM liquidity, and intent-based trading (e.g. CoW Swap).

4.1 | Roles

- **Owner** (DEFAULT_ADMIN_ROLE) — cold configuration: assets, protocols, payout rules, risk settings. Typically a hardware wallet or Safe.
- **Asset Manager** — hot execution within Guard and vault policy: trades, supply/stake, liquidity, intent orders. May be a human operator or an AI agent key.
- **Payout Operator** — proposes scheduled payments / sends within quota; owner approval binds content where required.

4.2 | Architecture

- **BittyV1VaultFactory** — activates one vault per owner via CREATE2 (salt = keccak256(owner)); owner is always msg.sender.
- **BittyV1Vault + BittyV1VaultDeFiFacet** — custody, scheduled payments, whitelist payouts; DeFi operations via facet/fallback and logic libraries.
- **BittyV1Guard** — registry and deprecation flags for assets, stablecoins, and protocol classes.
- **Protocol adapters** — per-vault minimal-proxy clones isolate positions; vault retains token custody for intent protocols.

4.3 | Security Model (as designed)

Day-to-day execution is constrained by on-chain Guard registration, vault-local protocol sets, `minimalBalances`, invest caps, and payout protection windows. Compromised DeFi frontends or a leaked asset-manager key should not be able to freely drain funds outside policy. Owner configuration remains the high-trust surface.

5 | Findings and Improvement Suggestions

Severity	Total	Acknowledged	Partially solved	Re-	Resolved
Critical	1	0	0		1
High	5	1	0		4
Medium	13	1	1		11
Low	13	4	0		9
Informational	11	3	0		8

Findings below consolidate the initial review (2026-07-06), follow-up notes (2026-07-13), and second-pass review (2026-07-21). Statuses reflect verification against the final commits in Section 3. Many issues were closed through later architecture changes (`RiskConfig`, trade limits, intent-only trading, factory redesign) in addition to point fixes.

5.1 | Critical

5.1.1 | C-01 — Vault Squatting via Permissionless Factory + Deterministic Addresses

Severity	Critical
Source	BittyV1VaultFactory.sol
Status	Resolved

■ Description

In the initial factory, `deployVault*` had no `msg.sender == owner` check; `CREATE2` salt was `keccak256(owner, name)`. An attacker could deploy a vault for a victim and assign themselves as asset manager before the victim activated the address. ETH sent to the predicted address was accepted via `receive()` prior to initialization.

■ Recommendation

Require the deployer to be the owner (or an owner-signed permit). Never fund a counterfactual vault before on-chain activation and role verification.

■ Client Response

Initially acknowledged as intentional so an AI-generated asset-manager key could activate a vault for a hardware-wallet / Safe owner. Later the factory was redesigned.

■ Status

Resolved. Final factory uses `activateVault*` with salt `keccak256(msg.sender)` and initializes the caller as owner only; one vault per owner.

5.2 | High

5.2.1 | H-01 — Factory initialize() Is Permissionless

Severity	High
Source	BittyV1VaultFactory.sol
Status	Resolved

■ Description

The first caller to `initialize()` could set `vaultImplementation`, `guardAddress`, and `wethAddress`. A deployment front-run could point the factory at malicious implementations.

■ Recommendation

Initialize in the constructor or restrict initialization to a known deployer.

■ Status

Resolved. `initialize` now requires `tx.origin == DEPLOYER` (constant deployer address for CRE-ATE2 factory flows).

5.2.2 | H-02 — Irreversible disableAddingProtocols Can Permanently Block Protocol Exit

Severity	High
Source	AssetManagerLogic.sol
Status	Resolved

■ Description

disableAddingProtocols() was irreversible. Exit paths still required the protocol in the vault's local set. Removing a protocol after locking adds permanently stranded positions (e.g. Aave supply with no exit path).

■ Recommendation

Provide exit-only paths that do not require Set membership, or allow re-adding / retaining protocols when clones/positions remain.

■ Status

Resolved. Vaults retain ability to exit from removed/deprecated protocols; related intent-protocol stranding addressed with H-03.

5.2.3 | H-03 — `removeIntentProtocols` Strands Active CoW / TWAP Orders

Severity	High
Source	AssetManagerLogic.sol; BittyV1Vault.isValidSignature
Status	Resolved

■ **Description**

Removing an intent protocol while orders were live broke cancel/clean paths and skipped the protocol in `isValidSignature`, while `forceApprove(vaultRelayer, max)` could remain.

■ **Recommendation**

Block removal while live orders exist, or allow cancel/clean without local-set membership.

■ **Status**

Resolved alongside the protocol-exit remediation (H-02).

5.2.4 | H-04 — CoW vaultRelayer Retains Unlimited Allowance After Orders End

Severity	High
Source	AssetManagerLogic / TradeLogic (_intentTrade, _executeCancel)
Status	Acknowledged

■ Description

Limit/TWAP flows approve the CoW vaultRelayer for `type(uint256).max`. Cancel/expiry deregistered orders but did not revoke allowance, leaving residual pull capacity if the relayer were compromised.

■ Recommendation

Revoke or decrease allowance on cancel; prefer per-order allowances over unlimited approval.

■ Client Response

Intent adapters (CoW Swap / previously UniswapX) are non-upgradeable and cannot seize user funds via the approval alone; unlimited approval is retained for gas efficiency for frequent traders. Only Guard-listed intent protocols may be added.

■ Status

Acknowledged (accepted residual risk / design trade-off).

5.2.5 | H-05 — YieldFeeTracker Taxes Principal as Yield Across Deposit/Withdraw Cycles

Severity	High
Source	Yield fee accounting (protocol-store / vault deliver path)
Status	Resolved
Round	2026-07-21 follow-up

■ Description

On withdrawal, full `grossAmount` (principal + yield) was booked into `totalWithdrawn`. After a full exit, `totalWithdrawn` permanently exceeded `totalDeposited`, so later pure-principal withdrawals were misclassified as earnings and charged the 10% yield fee.

■ Recommendation

Collapse the ledger onto remaining principal after each settlement, or use share/batch-based accounting so withdrawn principal cannot inflate future fee bases.

■ Status

Resolved. Yield fee accounting was removed from protocol-store (0145d2a).

5.3 | Medium

5.3.1 | M-01 — payReceiverAmount Does Not Enforce Trigger as Caller

Severity	Medium
Source	VaultLogic.sol
Status	Resolved

■ Description

`payReceiver` required `msg.sender == receiver.trigger` when set; `payReceiverAmount` only checked that a trigger was non-zero and did not verify the caller, allowing any address to trigger trigger-only payouts.

■ Recommendation

Require `msg.sender == receiver.trigger`.

■ Status

Resolved.

5.3.2 | M-02 — `payReceiverAmount` Ignores `amount` and Always Pays Full `receiver.amount`

Severity	Medium
Source	VaultLogic.sol
Status	Resolved

■ Description

The `amount` parameter was only an upper bound; transfer always used full `receiver.amount`.

■ Recommendation

Pass `amount` into the transfer path, or rename the API to full-payment semantics.

■ Status

Resolved.

5.3.3 | M-03 — Uniswap V3 Partial decreaseLiquidity Can Bypass Collect Fee

Severity	Medium
Source	UniswapV3Protocol.sol
Status	Resolved

■ Description

After partial `decreaseLiquidity`, accrued fees in `tokensOwed*` were misclassified as principal via `principalOwed*`, allowing subsequent `claimAMMFees` without the protocol fee cut.

■ Recommendation

Collect fees before partial decrease, or track principal vs fees explicitly.

■ Status

Resolved.

5.3.4 | M-04 — Guard Delisting Does Not Affect Existing Vault Asset Sets

Severity	Medium
Source	VaultLogic.checkAsset
Status	Acknowledged

■ **Description**

`checkAsset` reads vault-local sets only. Guard `removeAssets` / `removeStableCoins` does not propagate to deployed vaults, so managers can still trade locally listed tokens after Guard delisting.

■ **Recommendation**

Enforce live Guard registration in `checkAsset`, or document and enforce an off-chain sync policy.

■ **Client Response**

By design (“App Store” model): vaults retain previously selected assets; Guard controls what may be newly added.

■ **Status**

Acknowledged.

5.3.5 | M-05 — supply, stake, and addLiquidity Skip Vault Asset Allowlist

Severity	Medium
Source	BittyV1Vault / AssetManagerLogic / TradeLogic
Status	Partially Resolved

■ Description

Trade/intent paths called `checkAsset`; `supply`/`stake`/`addLiquidity` originally did not, allowing the asset manager to move any ERC20 held by the vault (including airdrops) into Guard-listed protocols.

■ Recommendation

Apply `checkAsset` to all asset-touching operations.

■ Client Response

Originally noted as by design for depositing out-of-Guard assets into Guard-listed protocols. Subsequently, `addLiquidity` was updated to call `VaultLogic.checkAsset` on both `token0` and `token1`.

■ Status

Partially Resolved. `addLiquidity` now enforces the vault asset allowlist. `supply` / `stake` still do not call `checkAsset` (protocol registration / deprecation checks only), consistent with allowing non-allowlisted vault balances into Guard-listed lending/staking.

5.3.6 | M-06 — Trade Rate Limits Removed; Only `minimalBalance` Remains

Severity	Medium
Source	AssetManagerLogic / TradeLogic
Status	Resolved

■ Description

Earlier `maxAmount` / `minimalDuration` rebalance limits were removed. A compromised asset manager could sell the entire balance above `minimalBalance` in one trade/order.

■ Recommendation

Restore per-trade caps/cooldowns, or document full trust in the asset manager for trade sizing.

■ Client Response

Restored via asset-manager settings and risk architecture: per-trade stablecoin size cap (`maxStableCoinPerTrade`), trade interval throttle (`interval` / `lastTradeTimestamp`), invest cap (`stableCoinInvestCap`), and optional full-access managers.

■ Status

Resolved. `_validateTrade` enforces these checks for restricted asset managers (full-access managers remain bounded only by `minimalBalance` / committed intent reservations by design).

5.3.7 | M-07 — updateReceiver Allows Immediate Payout via Past startTimestamp

Severity	Medium
Source	VaultLogic.sol
Status	Resolved

■ Description

`addReceiver` rejected past `startTimestamp`; `updateReceiver` did not, allowing immediate payout scheduling bypass.

■ Recommendation

Apply the same validation on update.

■ Status

Resolved (and later generalized to scheduled-payment update checks; see I-11).

5.3.8 | M-08 — Uniswap Swap Path Hops Not Validated Against Allowlist

Severity	Medium
Source	AssetManagerLogic (market sell/buy)
Status	Resolved

■ Description

Only `from/to` were allowlisted; encoded Uniswap multi-hop `path` intermediates were unchecked, enabling arbitrary intermediate routing.

■ Recommendation

Decode and validate every hop against Guard/vault assets.

■ Status

Resolved. Direct Uniswap swap paths were removed in favor of intent-only trading to reduce MEV/frontend attack surface (`protocol-store c07dbc8` / vault intent-only trading).

5.3.9 | M-09 — `_validateRebalance` / `minimalBalance` Does Not Cover Aggregate Intent Exposure

Severity	Medium
Source	AssetManagerTradeLogic / intent order placement
Status	Resolved
Round	2026-07-13 follow-up

■ Description

Limit/TWAP orders validated `minimalBalance` against full on-chain balance at placement, ignoring tokens already committed to other open orders. Multiple orders could each pass locally and collectively breach the floor (or fail at settlement).

■ Recommendation

Track committed sell amounts across open orders and subtract them from available balance in the floor check.

■ Status

Resolved via `committedIntentSell` reservations released on cancel/expiry.

5.3.10 | M-10 — cancel/cleanExpired Never Verify Owing Intent Protocol

Severity	Medium
Source	AssetManagerTradeLogic._executeCancel
Status	Resolved
Round	2026-07-21 follow-up

■ Description

Cancel paths checked clone existence and local order records but not that the supplied protocol owned the order. With ≥ 2 intent protocols, cancel via protocol B could clear local `committedIntentSell` while leaving protocol A's on-chain order live, undermining `minimalBalance`.

■ Recommendation

Store `owningProtocol` on each order record and require a match on cancel/clean.

■ Status

Resolved (`IntentProtocolMismatch` when `record.owningProtocol != intentProtocol`).

5.3.11 | M-11 — Selling Receipt Tokens on an AMM Sidesteps the Yield Fee

Severity	Medium
Source	Trade path vs yield-fee settlement
Status	Resolved
Round	2026-07-21 follow-up

■ Description

Yield fee applied only on protocol withdraw/unstake claim paths. Selling receipt tokens (aToken/stETH/sUSDS) via AMM converted yield-bearing exposure without hitting the fee tracker.

■ Recommendation

Route receipt-token exits through fee settlement, or forbid direct receipt-token trades.

■ Status

Resolved by removal of the yield-fee mechanism.

5.3.12 | M-12 — Lido Withdrawal Queue Dilutes / Dodges Yield Fee

Severity	Medium
Source	LidoV2Protocol unstake/claim + fee ledger
Status	Resolved
Round	2026-07-21 follow-up

■ Description

`unstake` did not reduce deposited principal on the ledger; staking during the queue window inflated `totalDeposited`, shrinking/zeroing earnings at claim.

■ Recommendation

Freeze principal share at unstake time and settle against that share at claim.

■ Status

Resolved by removal of the yield-fee mechanism.

5.3.13 | M-13 — Owner Approves Scheduled Payments by Id Only (Content Swap)

Severity	Medium
Source	VaultLogic.approveScheduledPayment
Status	Resolved
Round	2026-07-21 follow-up

■ Description

Approval keyed only on payment id. While pending, the proposer could `updateScheduledPayment` to change recipient/amount; the owner approving the id would authorize swapped content.

■ Recommendation

Bind approval to a content hash/version and invalidate on edit.

■ Status

Resolved. `approveScheduledPayment(id, expectedHash)` requires `keccak256(abi.encode(scheduledPayment)) == expectedHash`.

5.4 | Low

5.4.1 | L-01 — addIntentProtocols Bypasses disableAddingProtocols

Severity	Low
Source	AssetManagerLogic.sol
Status	Resolved

■ Description

Lending/staking/AMM adds used `onlyAddingProtocolsEnabled`; intent adds did not.

■ Status

Resolved (`addIntentProtocols` now uses the same modifier).

5.4.2 | L-02 — AMM LP Paths Do Not Lazy-Clone Adapters

Severity	Low
Source	AssetManagerLogic.sol
Status	Resolved

■ Description

Liquidity/fee paths required an existing clone; lending/staking lazy-cloned. Workaround required a dust trade first.

■ Status

Resolved.

5.4.3 | L-03 — `cleanExpiredOrders` Does Not Clear `activeTwapPerToken`

Severity	Low
Source	AssetManagerLogic / TradeLogic
Status	Resolved

■ Description

Expiry cleanup did not clear TWAP-per-token locks that `cancelTwap` cleared, blocking new TWAPs with `ActiveTwapExists`.

■ Client Response

Semantics clarified by renaming to `cleanExpiredLimitOrders` (limit vs TWAP paths).

■ Status

Resolved / clarified.

5.4.4 | L-04 — Intent Cancel Does Not Revoke Relay Allowance

Severity	Low
Source	<code>_executeCancel</code>
Status	Acknowledged

■ **Description**

Documented revoke behavior was not implemented (see also H-04).

■ **Status**

Acknowledged (same design rationale as H-04).

5.4.5 | L-05 — removeReceiver Leaves Stale lastReceiveTimestamps

Severity	Low
Source	VaultLogic.sol
Status	Resolved

■ Description

Re-adding a receiver under the same name inherited prior interval state and could unexpectedly hit `ReceiverInDuration`.

■ Status

Resolved.

5.4.6 | L-06 — updateReceiver Does Not Re-arm New-Receiver Protection

Severity	Low
Source	VaultLogic.sol
Status	Acknowledged

■ Description

After the initial cooling-off window, material receiver updates could take effect on the next payout without a second delay.

■ Client Response

By design: protection applies to new receivers only.

■ Status

Acknowledged.

5.4.7 | L-07 — Admin Transfer Delay Can Bypass Owner/Asset-Manager Separation

Severity	Low
Source	BittyV1Vault.sol
Status	Resolved

■ **Description**

Pending admin could be granted `ASSET_MANAGER_ROLE` during the transfer delay, collapsing role separation after acceptance.

■ **Status**

Resolved; vault ownership transfer was later removed entirely.

5.4.8 | L-08 — Uniswap swapExactOut Fee Computed on amountInMaximum

Severity	Low
Source	UniswapV3Protocol.sol
Status	Resolved

■ Description

Protocol fee charged on `amountInMaximum` rather than actual `amountIn`, overcharging when input was below the max.

■ Status

Resolved.

5.4.9 | L-09 — disableRebalanceUntilTimestamp Cannot Be Cleared; No Upper Bound

Severity	Low
Source	AssetManagerLogic.sol
Status	Resolved

■ Description

Asset manager could only extend the lock (including `type(uint256).max`); admin could not shorten/clear it.

■ Status

Resolved with a 4×365 days upper bound (later trading model further simplified).

5.4.10 | L-10 — Vault Implementation Lacks `_disableInitializers()`

Severity	Low
Source	BittyV1Vault.sol
Status	Acknowledged

■ Description

Implementation contract can be initialized directly (minimal-proxy hygiene). Clones remain the production path.

■ Status

Acknowledged (accepted residual hygiene risk).

5.4.11 | L-11 — Filled CoW Orders Remain in Registry Until Manual Deregister

Severity	Low
Source	CoWSwapV1Protocol.sol
Status	Acknowledged

■ Description

`activeOrders[hash]` not cleared on fill; `isValidSignature` remains valid until `validTo`. Settlement likely prevents double-fill; registry hygiene is weak.

■ Status

Acknowledged.

5.4.12 | L-12 — Vault remove*Protocols Blocks Exit While Guard Still Lists Protocol

Severity	Low
Source	AssetManagerLogic.sol
Status	Resolved

■ **Description**

Distinct from H-02: local removal alone blocked exits even when Guard still registered the protocol.

■ **Status**

Resolved (exit retained for removed protocols with remaining positions/clones).

5.4.13 | L-13 — Cancelling/Expiring Limit Orders Never Refunds Invest Cap

Severity	Low
Source	AssetManagerTradeLogic / invest accounting
Status	Resolved
Round	2026-07-21 follow-up

■ Description

Placing a stablecoin→asset limit/TWAP charged `stableCoinInvested` immediately; cancel/expiry released token reservations but not the invest cap, permanently consuming capacity. Similar inflation existed for unspent `marketBuy` max input.

■ Recommendation

Refund invest cap on cancel/expiry; book market buys against actual spend.

■ Status

Resolved (`_executeCancel` refunds `investedCounted`; intent-only trading path updated accordingly).

5.5 | Informational

5.5.1 | I-01 — deprecate*Protocols Can Mark Never-Registered Addresses

Severity	Informational
Source	BittyV1Guard.sol
Status	Resolved

■ Description

Deprecation flag could be set even when `Set.remove` was a no-op.

■ Status

Resolved.

5.5.2 | I-02 — Same Token Can Be Registered as Both Asset and Stablecoin

Severity	Informational
Source	BittyV1Guard; VaultLogic
Status	Acknowledged

■ Description

Independent registries allow dual registration; `addAssets` prefers the asset branch when both are set.

■ Client Response

Protocol managers are trusted to configure Guard correctly.

■ Status

Acknowledged.

5.5.3 | I-03 — Aave Withdraw Dust / Rounding

Severity	Informational
Source	AaveV3Protocol.sol
Status	Resolved

■ Description

aToken balance can be slightly below supplied amount; exact full withdraw of the original amount may revert.

■ Status

Resolved (`type(uint256).max` full-exit support).

5.5.4 | I-04 — Lido stETH Rounding on Precise Unstake

Severity	Informational
Source	LidoV2Protocol.sol
Status	Resolved

■ Description

Same rounding class as Aave for precise unstake amounts.

■ Status

Resolved.

5.5.5 | I-05 — Sky `getStakedBalance` vs PSM `tout` on Full Exit

Severity	Informational
Source	SkyV1Protocol.sol
Status	Resolved

■ **Description**

`getStakedBalance()` returned gross USDC value; `unstake(getStakedBalance())` could revert when PSM `tout` > 0.

■ **Status**

Resolved.

5.5.6 | I-06 — removeAssets/checkAsset Use NotRegistered for Vault-Local Misses

Severity	Informational
Source	VaultLogic.sol
Status	Acknowledged

■ Description

Error naming reads as Guard rejection when the address is merely absent from the vault-local set.

■ Client Response

Prefer `NotRegistered` when users attempt to add assets not listed in Guard.

■ Status

Acknowledged.

5.5.7 | I-07 — RECEIVER_NEW_PROTECTION_MIN Documented but Not Enforced

Severity	Informational
Source	VaultLogic.sol / RiskConfig
Status	Resolved

■ Description

Comments originally referenced a minimum protection duration; only the maximum was enforced, and default 0 disabled protection entirely.

■ Client Response

Addressed via **RiskConfig** / risk levels: **Standard** and **High** seed non-zero **scheduledPaymentProtection** and **whitelistedProtection** (e.g. 3-day standard timelock). Immutable scheduled-payment lock also clamps the window to at least **STANDARD_RISK_TIMELOCK**.

■ Status

Resolved through **RiskSetting**. (**RiskControlLevel.Zero** remains an explicit no-controls profile for testing.)

5.5.8 | I-08 — Missing `onlyInitialized` on Selected Entrypoints

Severity	Informational
Source	VaultLogic / AssetManagerLogic
Status	Resolved

■ Description

Some peer functions lacked `onlyInitialized` while others required it.

■ Status

Resolved.

5.5.9 | I-09 — Guard Admin Assigned to tx.origin at Deploy

Severity	Informational
Source	BittyV1Guard.sol
Status	Resolved

■ Description

Constructor originally granted admin/manager roles to whatever `tx.origin` was at deploy, which is fragile for contract/meta-tx deployers.

■ Client Response

Hardened: admin and manager roles are granted to a constant `DEPLOYER` address via `AccessControlDefaultAdminRule(DEPLOYER)`, and construction requires `tx.origin == DEPLOYER`. Vault activation ownership uses `msg.sender`.

■ Status

Resolved. (`tx.origin` remains only as the `CREATE2`-factory deployer gate against the fixed `DEPLOYER` constant, not as an open-ended admin assignment.)

5.5.10 | I-10 — isValidSignature Silently Ignores Failing Intent Clones

Severity	Informational
Source	BittyV1VaultDeFiFacet.isValidSignature
Status	Acknowledged

■ Description

Intent clone `isValidSignature` calls are wrapped in `try/catch`; broken/malicious clones fail silently. Trust depends on Guard registration quality.

■ Status

Acknowledged (still present in final code; acceptable given Guard gating).

5.5.11 | I-11 — updateScheduledPayment Missing startTimestamp Check

Severity	Informational
Source	VaultLogic.sol
Status	Resolved
Round	2026-07-21 follow-up

■ Description

addScheduledPayment enforced `startTimestamp >= block.timestamp`; updateScheduledPayment did not (consistency gap; payout accrual still rejects future timestamps).

■ Status

Resolved.

5.6 | Positive Security Properties Observed

- Clear separation of Owner / Asset Manager / Payout Operator responsibilities.
- Guard registration and deprecation gates for protocols and (on add) assets.
- Per-vault protocol clones with vault-held custody for intent trading.
- Content-hash binding for scheduled-payment owner approval.
- `committedIntentSell` reservations to protect `minimalBalance` across open orders.
- Invest-cap refunds on cancel/expiry.
- Configurable new-receiver / scheduled-payment protection windows.

6 | Appendix

6.1 | Appendix I: Severity Categories

Severity	Description
Critical	Issues that enable direct fund loss or permanent lock without realistic admin recovery under expected usage. All critical severity issues should be resolved.
High	Issues that are highly exploitable security vulnerabilities. It may cause direct loss of funds / permanent freezing of funds. All high severity issues should be resolved.
Medium	Issues that are only exploitable under some conditions or with some privileged access to the system. Users' yields/rewards/information is at risk. All medium severity issues should be resolved unless there is a clear reason not to.
Low	Issues that are low risk. Not fixing those issues will not result in the failure of the system. A fix on low severity issues is recommended but subject to the clients' decisions.
Information	Issues that pose no risk to the system and are related to the security best practices. Not fixing those issues will not result in the failure of the system. A fix on informational issues or adoption of those security best practices-related suggestions is recommended but subject to clients' decision.

6.2 | Appendix II: Status Categories

Status	Description
Unresolved	The issue is not acknowledged and not resolved.
Partially Resolved	The issue has been partially resolved.
Acknowledged	The Finding / Suggestion is acknowledged but not fixed / not implemented.
Resolved	The issue has been sufficiently resolved.

7 | Disclaimer

Verilog Solutions receives compensation from one or more clients for performing the smart contract and auditing analysis contained in these reports. The report created is solely for Clients and published with their consent. As such, the scope of our audit is limited to a review of code, and only the code we note as being within the scope of our audit is detailed in this report. It is important to note that the Solidity code itself presents unique and unquantifiable risks since the Solidity language itself remains under current development and is subject to unknown risks and flaws. Our sole goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies. Thus, Verilog Solutions in no way claims any guarantee of security or functionality of the technology we agree to analyze.

In addition, Verilog Solutions reports do not provide any indication of the technology's proprietors, business, business model, or legal compliance. As such, reports do not provide investment advice and should not be used to make decisions about investment or involvement with any particular project. Verilog Solutions has the right to distribute the Report through other means, including via Verilog Solutions publications and other distributions. Verilog Solutions makes the reports available to parties other than the Clients (i.e., "third parties") – on its website in hopes that it can help the blockchain ecosystem develop technical best practices in this rapidly evolving area of innovation.